

Guardian Shell Engineering Architecture Review

Prepared for engineering review Date: March 13, 2026 Audience: security engineers, platform engineers, systems engineers

Executive Summary

Guardian Shell is an agent-centric Linux security system that combines:

- eBPF tracepoints
- BPF-LSM enforcement
- cgroup-backed agent identity
- userspace policy management
- dashboard, alerting, and approval workflows

It is already a credible architecture for agent governance, but its current trust boundary is not yet as strong as it should be for adversarial LLM-agent behavior.

The primary technical issue is that core policy still depends too heavily on path-based, tracepoint-first reasoning. The right direction is a layered evolution toward:

1. cgroup-first identity
2. LSM-first file and exec decisions
3. stronger executable identity for high-assurance mode
4. seccomp for dangerous syscall classes
5. session-scoped policy state
6. optional stronger isolation tiers for high-risk deployments

Reader Orientation

This document is written to be useful to two audiences at the same time:

- engineers who already understand Linux security and eBPF
- junior engineers who are new to Linux internals, cgroups, tracepoints, and LSM-based enforcement

How to read it:

1. the first sections explain the current system in concrete product and runtime terms

2. later sections explain why the current design is only a transitional trust model
3. the technical primer near the end defines the Linux and eBPF concepts in plain language

If you are new to Linux security, the most important mental model is:

```
launcher creates the agent session
daemon loads policy and manages state
eBPF observes kernel activity
LSM hooks enforce security decisions
dashboard and IPC make the system operable by humans
```

Technical Primer

What is eBPF?

eBPF lets verified programs run inside the Linux kernel.

What is the verifier?

The verifier proves safety properties such as bounded execution and safe memory access.

What are tracepoints?

Kernel instrumentation points used for syscall and scheduler event capture.

How they work:

1. the kernel exposes named instrumentation points
2. an eBPF program is attached to one of those points
3. when that kernel event occurs, the eBPF program runs
4. the program can read the event context, update maps, and emit an event

Example in Guardian:

- sys_enter_openat sees the raw pathname argument as the process enters the syscall
- sched_process_fork sees parent/child creation and supports process-tree tracking

Why they are useful:

- easy access to syscall arguments
- strong observability
- good fit for monitoring and for "early" policy logic

Why they are not enough by themselves:

- they observe the request before the kernel finishes resolving the real object
- some alternate interfaces can avoid the same observation path
- they are better as an observability layer than as the final trust anchor

What is LSM?

LSM means Linux Security Modules.

It is the kernel security framework that lets Linux call security checks at important operations such as opening files, executing programs, or creating network connections.

How it works:

1. the Linux kernel defines security hook points
2. security modules attach logic to those hook points
3. when a protected operation happens, the kernel calls that logic
4. the security logic allows or denies the operation

Examples of LSM-based systems include AppArmor, SELinux, Landlock, and BPF-LSM programs.

Why it matters in Guardian:

- Guardian uses the LSM framework as the kernel enforcement path
- this is where file and exec blocking happen
- moving more of Guardian's decision-making into LSM is a key part of the future architecture

What are LSM hooks?

Kernel security decision points such as `file_open` and `bprm_check_security`.

How they work:

1. the kernel reaches a security-relevant operation
2. it invokes the corresponding LSM hook
3. the attached security program can inspect the kernel object context
4. it returns allow or deny

Example in Guardian:

- `file_open` can block file access
- `bprm_check_security` can block execution before the new program starts

Why they matter:

- they sit on the enforcement path

- they are closer to the real kernel object than syscall-entry tracepoints
- they are the right place for final allow or deny decisions

What is BPF-LSM?

Attaching eBPF programs to LSM hooks.

In practical terms, BPF-LSM lets Guardian run custom eBPF logic inside kernel security decision points instead of using only traditional built-in MAC modules.

What are BPF maps?

Shared kernel/user-space key-value structures used for:

- watched identities
- enforcement identities
- allow/deny rules
- pending decisions
- event transport support

What is IPC?

Inter-process communication.

In Guardian Shell it is primarily used for local coordination between:

- guardian-launch
- guardian
- control utilities

How it works in Guardian:

1. the daemon listens on a local Unix socket
2. another local component connects to that socket
3. it sends a structured request such as:
 - register an agent session - list agents - stop an agent - request permission - grant temporary access
4. the daemon validates the request and updates in-memory state or BPF-backed policy state
5. the daemon sends a structured response back to the caller

Example flows:

- guardian-launch uses IPC to register a newly created cgroup-backed agent session

with the daemon

- guardian-ctl uses IPC to request a temporary grant or ask the daemon to stop an agent
- permission requests use IPC so an agent can ask for access and wait for the daemon's approval decision

Why this matters:

- kernel eBPF programs do enforcement and event capture
- user-space components still need a reliable control channel to coordinate policy, approvals, and lifecycle events
- IPC is the glue between the launcher, daemon, CLI, and dashboard-facing workflows

What is a Unix socket?

The local IPC transport endpoint used by the control plane.

What is a perf event array?

The mechanism used to send structured events from kernel space to user space.

What is a cgroup?

A kernel process-grouping mechanism used here for both resource control and session identity.

How it works in Guardian:

1. guardian-launch creates a dedicated cgroup for the agent session
2. the target process is moved into that cgroup before it starts normal work
3. children inherit membership automatically
4. the kernel-side eBPF program reads the current cgroup ID during later activity
5. policy is applied to the whole session, not just one process name

What are TGID and comm?

- TGID: process-family identifier
- comm: short process name

Both are useful, but weaker than cgroup-backed identity.

How TGID and process-tree tracking work:

1. Guardian identifies an initial process family by TGID
2. fork tracepoints observe child creation

3. children are added to tracking maps
4. future file or exec activity is matched against that tracked family

How comm fallback works:

1. Guardian reads the task's short process name
2. it compares that name to watched names
3. if there is a match, policy is applied even without cgroup or TGID context

Why fallbacks happen:

- the process may not have been launched through Guardian
- the system may be in discovery or compatibility mode
- operators may know the agent only by its current runtime name at first

Trust meaning:

- cgroup: strongest session identity
- TGID/process tree: useful process-family identity
- comm: weakest compatibility identity

What are openat, execve, and connect?

Core syscalls for:

- file access
- program execution
- outbound network connection

What is TOCTOU?

Time-of-check to time-of-use. Relevant whenever a decision is made in one kernel context and consumed later.

What is canonical path resolution?

Resolving the real target path after symlink, traversal, and mount effects.

What is an inode?

The kernel's file object metadata identity. Often a stronger basis than names alone.

What is seccomp?

Seccomp is a Linux syscall-filtering mechanism used to reduce which syscalls a process may use.

How it works:

1. a process starts with a seccomp filter
2. each syscall is checked against that filter
3. the kernel can allow it, deny it, or trap it depending on policy

Why it matters here:

- seccomp does not replace file or exec policy
- seccomp is good at blocking dangerous syscall classes such as ptrace, unshare, setns, or io_uring_setup
- it reduces the number of alternate interfaces an adaptive agent can use while file and exec controls do the more context-aware work

Current System Overview

Current implemented capabilities

The current codebase includes:

- sys_enter_openat, openat2, and legacy open tracing
- file_open LSM-based file blocking
- sys_enter_execve tracing
- bprm_check_security LSM-based exec blocking
- sys_enter_connect tracing for outbound network visibility
- sched_process_fork and exit tracking
- cgroup, TGID, and comm-based identity
- cgroup launcher and IPC registration
- alerting, dashboard, metrics, CLI, and permission management

Current architecture in concrete terms

The current Guardian Shell architecture is best described as a hybrid system with four cooperating parts:

1. a launcher and registration path
2. a long-running daemon and policy engine
3. kernel-resident eBPF programs
4. a user-visible operations plane

This is important because Guardian is not only an eBPF program. The security story

depends on all four parts working together.

Current architecture: end-to-end flow

For a new engineer, the simplest way to understand Guardian is as one end-to-end runtime flow:

1. guardian-launch starts an agent in a dedicated cgroup
2. guardian-launch registers that session with the daemon over IPC
3. guardian daemon loads eBPF programs and fills BPF maps with identity and policy state
4. agent performs file, exec, or network activity
5. kernel eBPF code identifies the acting session or process
6. tracepoints observe the request and emit rich telemetry
7. LSM hooks enforce file or exec blocking when enforcement is active
8. daemon receives events, stores them, alerts on them, and exposes them to the dashboard
9. human operators can inspect, approve, deny, or temporarily grant access

That is the current product shape: a launcher, a daemon, kernel programs, and a dashboard/control plane acting together.

High-level architecture

User space

- guardian daemon
 - config parsing
 - BPF loading
 - BPF map population
 - event processing
 - alerting / dashboard / permissions

- guardian-launch
 - cgroup creation
 - resource-limit setup
 - cgroup registration via IPC
 - move self to cgroup
 - exec target command

Kernel space

- tracepoints
 - open/openat/openat2
 - execve
 - connect
 - fork/exit

- LSM hooks
 - file_open
 - bprm_check_security

- BPF maps
 - identity maps
 - allow/deny policy maps
 - pending deny maps
 - event buffers

Current architecture by responsibility

1. guardian-launch

This is the secure entry point for high-assurance sessions.

It is responsible for:

- creating the agent cgroup
- setting cgroup controller limits
- deriving the kernel-visible cgroup identity
- registering the session with the daemon
- entering the cgroup before the target command starts

This matters because identity is strongest when it is established before the agent begins executing.

2. guardian daemon

The daemon is the policy and control-plane center.

It is responsible for:

- reading and validating configuration
- loading BPF programs
- attaching tracepoints and LSM hooks
- populating and updating BPF maps
- consuming event streams
- applying user-space policy checks and approval logic
- serving dashboards, alerts, and metrics

This is where Guardian differs from many research-style eBPF tools. The daemon is not incidental. It is part of the product.

3. guardian-ebpf

The kernel side is responsible for:

- identifying the current process or agent session
- reading relevant syscall or hook context
- evaluating kernel-side subsets of policy
- setting pending deny flags where current architecture still uses two-phase enforcement

- blocking in LSM hooks
- sending structured events to user space

4. Supporting control surfaces

These include:

- IPC over a Unix socket
- dashboard routes and database storage
- alert sinks such as JSON, webhook, Slack, email, metrics
- permission request handling

This matters because the product's security value is partly enforcement, but partly decision support and auditability.

Short dashboard architecture summary

The dashboard is a lightweight server-rendered operations UI, not a separate SPA.

Current implementation:

- backend HTTP server: Axum
- HTML templating: Askama
- frontend interaction: Alpine.js
- partial page actions: htmx
- live updates: Server-Sent Events using the browser EventSource API

How it works at a high level:

1. the daemon starts an embedded Axum server
2. Askama renders HTML pages on the server
3. Alpine.js handles small client-side state such as pending permission banners and countdown timers
4. htmx submits actions like approve, deny, reload config, stop agent, and grant access without a full page reload
5. one shared SSE connection streams live events and permission updates to the browser

This is intentionally simple. The dashboard is designed as an operator console tightly coupled to the daemon, not as a large frontend application with a separate API gateway and build system.

Example: what happens when an agent tries to read a secret file

Scenario:

- agent launched through guardian-launch
- policy allows /workspace/project/**
- policy denies /home/dev/.ssh/**
- agent runs cat /home/dev/.ssh/id_rsa

Current flow:

1. agent process is already inside its dedicated cgroup
2. tracepoint on open/openat sees the raw requested path
3. kernel code identifies the process by cgroup, TGID, or comm
4. kernel code evaluates current map-based file policy
5. if denied, it writes PENDING_DENY for that pid/tgid
6. file_open LSM hook runs
7. LSM hook sees PENDING_DENY and returns an access error
8. event is emitted to user space
9. daemon logs, alerts, stores, and displays the denial

This example shows both the power and the weakness of the current design:

- power: the access can be blocked in kernel space
- weakness: the core decision was still made from the raw requested path rather than the final kernel-resolved object

Example: what happens when an agent runs a command

Scenario:

- exec policy allows /usr/bin/git
- exec policy denies /usr/bin/curl
- the agent attempts curl https://example.com

Current flow:

1. execve tracepoint captures the requested executable path
2. identity is resolved using cgroup/TGID/comm logic
3. exec policy is checked against kernel maps
4. if denied, PENDING_EXEC_DENY is written
5. bprm_check_security LSM hook runs
6. the pending entry is consumed
7. execution is blocked
8. event is sent to user space for logging and alerting

Again, this is useful today, but weaker than direct binary-identity enforcement.

Agent Identity Design

Why identity is hard

An LLM agent is rarely one stable process with one unique name.

Examples:

- Claude Code may appear as claude and spawn node, bash, git
- Aider may appear as python3
- another Python-based agent may also appear as python3

So identity must solve:

1. distinguishing agents using the same runtime
2. carrying policy across child processes
3. reducing spoofability
4. remaining operationally usable for real developer workflows

Identity layers in the current design

1. comm process name

Pros:

- simple
- good for discovery
- easy fallback mode

Cons:

- 16-byte limit
- ambiguous for common runtimes
- weak against spoofing
- poor fit for subprocess-heavy workloads

Concrete example:

Policy watches: python3

Problem:

aider	-> python3
open-interpreter	-> python3
custom agent	-> python3

Result:

a name-only policy cannot tell these sessions apart

2. TGID and child-process tracking

Pros:

- better process-family coverage
- improves over name-only matching

Cons:

- transient identifiers
- lifecycle bookkeeping
- still weaker than session-backed identity

Concrete example:

```
main agent pid/tgid: 4200
agent forks bash:    4210
bash runs git:       4215
```

If only the original process is watched, git may escape governance.
TGID/child tracking closes part of that gap.

3. Cgroup identity

This is the strongest current identity method.

Pros:

- session-scoped
- inherits automatically to children
- harder to spoof
- aligns with resource governance and lifecycle management

This should be considered the target default secure mode.

Concrete example:

```
guardian-launch --name claude-fix -- claude
```

Processes later seen:

```
  claude
  bash
  git
  pytest
  python3
```

All stay attributable to the same cgroup-backed agent session.

Cgroup implementation details

1. guardian-launch --name <agent> -- <cmd>
2. create /sys/fs/cgroup/guardian/<agent>--<pid>

3. enable subtree controllers
4. apply memory/pids/cpu limits if configured
5. stat cgroup directory and derive inode-based cgroup ID
6. send IPC registration to daemon
7. daemon updates WATCHED_CGROUPS / ENFORCE_CGROUPS / defaults
8. launcher writes its PID into cgroup.procs
9. launcher execs target command
10. descendants inherit cgroup membership
11. eBPF reads bpf_get_current_cgroup_id()
12. policy is applied using cgroup-first identity

What fallback means in the identity model

Guardian prefers identity signals in this order:

1. cgroup identity
2. TGID and child-process tracking
3. comm process name

Fallback exists because not every process starts inside guardian-launch, not every deployment begins with full cgroup-backed registration, and operators may need a compatibility or discovery mode before moving to the strongest identity model.

If cgroup identity is present

This is the intended secure mode.

What happens:

1. the launcher creates and registers a dedicated cgroup
2. the daemon inserts that cgroup ID into watched and enforce maps
3. every child process remains in that cgroup
4. kernel checks match this session by cgroup before considering weaker identity signals

Example:

```
guardian-launch --name claudex-fix -- claudex
```

Later process tree:

```
claudex
-> bash
-> git
-> python3
```

All remain attributable to the same cgroup-backed session.

If TGID fallback happens

This means Guardian does not have a cgroup-backed session identity for the process, so

it falls back to process-family tracking.

How it works:

1. Guardian records the watched process TGID
2. sched_process_fork tracepoints observe child creation
3. child TGIDs are inserted into tracking maps
4. later policy checks match if the current process belongs to that tracked family

Example:

```
Initial watched process:
python3 (tgid 4200)
```

```
Children later created:
bash    (tgid 4210)
git     (tgid 4215)
```

Guardian still treats those descendants as part of the same watched process family.

What this means operationally:

- Guardian can still follow subprocess trees
- this is stronger than name-only matching
- identity is still transient because TGIDs are live process identifiers, not durable session containers
- correct coverage depends on fork tracking and cleanup working correctly

If process-name fallback happens

This is the weakest mode and should be understood as compatibility or discovery behavior, not the strongest security mode.

How it works:

1. kernel code reads the task's comm
2. that short name is checked against watched and enforce name maps
3. if the name matches, policy is applied

Example:

```
Watched by name:
python3
```

```
Possible matches:
aider          -> python3
open-interpreter -> python3
custom tool    -> python3
```

What this means operationally:

- Guardian can still discover or monitor likely agent processes
- unrelated processes may collide on the same name
- the name is short and can often be changed or spoofed
- subprocess-heavy workloads are harder to reason about cleanly

The engineering takeaway is simple: cgroup mode is the target secure mode, TGID fallback is a useful intermediate mode, and process-name fallback is the weakest but still useful for compatibility and initial rollout.

Identity priority in kernel logic

Conceptually:

```
if current cgroup is watched:
    watched
else if current TGID or tracked child is watched:
    watched
else if current comm is watched:
    watched
else:
    ignore
```

Implementation note:

This pseudocode describes the intended reasoning order. In the current code, cgroup identity is clearly the strongest signal, while TGID, child tracking, and comm-based matching are combined as fallback mechanisms in helper logic. Default-action handling is not yet fully symmetric across all three identity layers, which is another reason the document treats cgroup-backed session identity as the long-term center of gravity.

Identity issues encountered

1. multiple agents sharing python3 or node
2. child-process escape from name-only policy
3. weak trust in process-name identity
4. discovery gaps without explicit launch registration
5. operational need to keep the identity model understandable to users

Issues faced in more detail

Issue A: Name collisions among real agents

Problem:

The actual ecosystem of AI tools often reuses generic runtimes.

Examples:

- Aider and a custom Python agent both become python3
- Claude-related helpers may appear as node
- different Electron-based tools may look similar in process listings

Why this matters:

- policy may attach to the wrong process family
- monitoring output becomes hard to interpret
- one agent may accidentally inherit another agent's visibility

Current answer:

- prefer cgroup-backed launch mode
- use comm-based mode only as compatibility or discovery mode

Issue B: The real action often happens in subprocesses

Problem:

The agent prompt loop itself may not directly perform the risky action. Instead, it spawns a shell or helper.

Example:

```
agent prompt process -> bash -> cat ~/.aws/credentials
```

If identity sticks only to the top-level process name, the dangerous access is missed.

Current answer:

- sched fork tracking
- TGID and child maps
- cgroup inheritance as the cleanest solution

Issue C: Discovery after-the-fact is weaker than explicit registration

Problem:

If the daemon has to discover agents only by scanning /proc, then:

- agents started later may be missed for some interval
- attribution is inferential rather than explicit
- race windows become harder to reason about

Current answer:

- rescans still exist as a fallback
- guardian-launch makes the session boundary explicit before agent work starts

Issue D: Identity must work across monitoring and enforcement

Problem:

If the identity model used for logs differs from the model used for blocking, operator trust degrades.

Current answer:

- the kernel identity helpers prioritize the same layered checks used across event types
- cgroup identity is treated as the strongest signal in both monitoring and enforcement paths

Current solution

The present design solves these pragmatically through:

- launcher-managed cgroup sessions
- TGID and child tracking
- comm as compatibility mode rather than strong identity
- partially per-agent behavior, but not yet fully isolated per-agent kernel policy state

Why cgroups are the right center of gravity

For LLM agents, the best identity unit is not "the executable" and not "the current process name." It is the agent session.

Cgroups match that need well because:

1. they cover the whole subprocess family
2. they align with resource controls
3. they provide a durable kernel-visible identity
4. they fit lifecycle actions like stop, list, and audit

This is why the future architecture should make cgroup-backed identity the explicit secure default, not just one available option.

Current Enforcement Model

File access flow

```
open/openat/openat2 tracepoint
-> read raw filename
-> identify process
-> evaluate allow/deny maps
-> if denied, write PENDING_DENY
file_open LSM hook
-> consume PENDING_DENY
-> return -EACCES when present
```

Example:

Policy:

```
allow /workspace/project/**
deny  /home/dev/.ssh/**
```

Agent action:

```
open("/home/dev/.ssh/id_rsa")
```

Result today:

```
deny decision is made from the tracepoint-visible path
LSM hook blocks using the pending flag
```

Exec flow

```
execve tracepoint
-> read raw executable path
-> evaluate exec policy
-> if denied, write PENDING_EXEC_DENY
bprm_check_security LSM hook
-> consume PENDING_EXEC_DENY
-> return denial when present
```

Example:

Policy:

```
allow /usr/bin/git
deny  /usr/bin/curl
```

Agent action:

```
execve("/usr/bin/curl", ...)
```

Result today:

```
tracepoint sets pending exec deny
LSM hook blocks execution
```

Why this design exists

The architecture separates:

- tracepoint-friendly capture of syscall arguments
- LSM-based enforcement

This is pragmatic because raw syscall arguments are easier to capture at tracepoints

than to reconstruct inside the LSM hook.

It is a reasonable transitional architecture, especially when building a product incrementally. But it is not the ideal end-state architecture for adversarial agent security.

Strengths

1. good observability
2. simple path-policy semantics for current product needs
3. incremental compatibility across kernels and deployments
4. keeps enforcement in kernel while preserving a rich user-space product plane

Why the current architecture was a sensible starting point

From an engineering perspective, the current architecture likely emerged because it optimizes for practical implementation constraints:

1. tracepoints are easier for reading syscall arguments
2. path strings are easier to reason about initially than inode or object identity
3. user-space policy and dashboard work can proceed in parallel with kernel enforcement work
4. fallback behavior is easier when some advanced kernel features are unavailable

This is worth stating explicitly. The current design is not irrational. It is a pragmatic intermediate architecture.

Weaknesses

1. trust boundary remains tracepoint-first for core policy
2. path identity is weak against aliasing
3. pending-map handoff complicates correctness reasoning
4. some policy state is still global rather than fully per session

Why these weaknesses matter more for LLM agents than for ordinary workloads

Traditional workloads usually do not actively search for bypasses.

LLM agents may:

- try alternate file names
- try helper binaries
- try subprocess chains
- try alternate system interfaces

- ask for policy changes when blocked

That means any weak trust anchor is more likely to be stressed in practice.

Comparison With Veto

Guardian Shell and Veto solve different primary problems.

Veto

Primary question:

"Should this binary run at all?"

Core strengths:

- binary identity via content hash
- strong pre-exec trust anchor
- robust against rename, copy, and symlink-based executable bypasses

Guardian Shell

Primary question:

"What can this agent session access?"

Core strengths:

- file governance
- agent/session-oriented governance
- approval workflow
- observability and operations plane
- better fit for interpreter-heavy agent behavior

Important current caveat:

The product is agent/session-oriented, but some kernel policy maps are still shared globally rather than fully isolated per session. That means the control model is pointed in the right direction, but not yet in its final strongest form.

Engineering comparison

+-----+			
Dimension	Guardian Shell	Veto	
+-----+			
Primary unit	agent/session	executable binary	
Identity anchor	cgroup/TGID/comm	content hash	

File governance	yes	limited	
Exec governance	path-based today	stronger	
Approval workflow	yes	not core focus	
Observability plane	rich	more enforcement-centric	
Best fit	agent resource governance	strong binary execution trust	
+-----+-----+-----+-----+			

Practical conclusion

Veto is stronger at execution identity.

Guardian Shell is stronger as an agent-governance platform.

For high-assurance deployments, a Veto-like execution trust primitive and Guardian-like resource governance can coexist.

Example scenarios

Scenario 1: copied binary bypass

Denied path:

/usr/bin/curl

Agent action:

```
cp /usr/bin/curl /tmp/.x
/tmp/.x https://example.com
```

Veto-style binary identity is stronger here.

Scenario 2: interpreter-heavy task

Allowed binary:

/usr/bin/python3

Actual risk:

python reads secrets from ~/.ssh or ~/.aws

Guardian-style file governance is stronger here.

Scenario 3: human approval workflow

Agent asks for:

```
/workspace/legacy-migration/**
duration: 20 minutes
```

Guardian is the more natural product fit because approval and audit are first-class product features.

Current Security Limitations

1. Path aliasing and canonicalization gaps

The main problem is that policy still depends heavily on raw path strings seen at syscall entry.

Examples:

- /proc/self/root/etc/shadow
- symlink indirection
- bind mounts
- hard links

Userspace normalization is useful, but not a full substitute for canonical or inode-aware enforcement.

Example scenario:

Denied:
 /etc/shadow

Agent tries:
 /proc/self/root/etc/shadow

If policy relies on the raw requested string, equivalence with the real target may be incomplete.

2. Two-phase decision and enforcement

The PENDING_DENY and PENDING_EXEC_DENY handoff model adds:

- timing assumptions
- cleanup requirements
- harder reasoning across edge cases

Example scenario:

tracepoint decides deny
LSM later consumes pending state

Any architecture that splits "decide" and "enforce" must reason carefully about races, cleanup, and hook ordering assumptions.

3. Exec policy is path-based, not binary-identity-based

This is materially weaker against adaptive agents.

Example scenario:

Denied:
 /usr/bin/wget

Agent:

```
cp /usr/bin/wget /tmp/helper  
/tmp/helper
```

4. Interpreters compress many actions behind one allowed binary

Allowing python3 or node is not meaningful by itself unless file and resource governance is strong underneath it.

Example scenario:

Allowed:
/usr/bin/python3

But script reads:
~/.aws/credentials
~/.ssh/id_rsa

5. Network policy is behind file and exec maturity

Outbound connect visibility exists, but enforcement depth is not yet at the level many security teams will expect.

Example scenario:

Agent cannot read a local secret file
Agent instead tries to download a remote helper or exfiltrate data over HTTPS

6. Alternate surfaces such as io_uring

Tracepoint-centric observability must always account for alternate interfaces that do not pass through the same path.

Example scenario:

Expected observation path:
regular syscall tracepoint

Alternate path:
io_uring-based operation

7. Shared host-kernel trust model

Even a strong BPF-based design is still a host-level control model, not the same as VM isolation.

Example scenario:

High-risk third-party agent plugin runs on the same host kernel as developer workloads.
This is where stronger isolation tiers become necessary.

Summary of limitations

The current architecture is good enough to demonstrate product value and meaningful

policy enforcement, but it is not yet the final form needed for stronger adversarial-agent claims.

Recommended Future Architecture

The correct future design is an evolution, not a rewrite.

Design principles

1. keep the existing user-space product surface
2. strengthen the kernel trust anchor
3. make cgroup-backed session identity primary
4. separate agent identity from resource identity
5. support multiple assurance tiers

Proposed layered architecture

Product layer

- dashboard
- alerts
- approvals
- audit trail
- policy authoring

Agent identity layer

- cgroup as primary session identity
- launcher-managed lifecycle
- per-agent policy object

Kernel enforcement layer

- LSM file_open with canonical or inode-aware decisions
- LSM bprm_check_security with stronger exec identity
- seccomp for dangerous syscalls
- network enforcement growth over time

Host / isolation layer

- AppArmor or SELinux baseline
- optional namespace/container isolation
- optional microVM / VM high-assurance tier

Why the proposed architecture is better in engineering terms

The proposed architecture is better not because it is more complex, but because it moves each decision to the layer that can evaluate it more correctly.

Examples:

- file identity decisions move closer to the real file object

- dangerous syscall classes move to seccomp
- agent identity is anchored to cgroups rather than names
- observability stays rich, but is no longer the same thing as the trust anchor

Current vs proposed

Area	Current Guardian Shell	Proposed Future Guardian Shell
Primary file decision	tracepoint-first, path string	LSM-first, canonical/inode-aware
File blocking	pending deny handoff	decision and block in one place
Exec control	path-based exec policy	stronger executable identity
Agent identity	cgroup/TGID/comm	cgroup-first
Network	connect monitoring	policy-aware enforcement
Dangerous syscalls	limited	seccomp standard layer
Policy isolation	partly global maps	per-session policy state

Proposed file flow

```
Agent open()
-> LSM file_open receives real file object
-> resolve canonical path or inode/device identity
-> evaluate per-agent rules directly in the LSM path
-> allow or deny
-> emit observability event
```

Example scenario:

```
Denied:
/home/dev/.ssh/**
```

```
Agent tries:
/tmp/key-link -> /home/dev/.ssh/id_rsa
open("/tmp/key-link")
```

In the proposed design, the file decision should be made against the resolved target identity rather than only the requested string.

Proposed exec flow

```
Agent execve()
-> LSM bprm_check_security
-> resolve executable identity
-> standard mode: canonical path policy
-> high-assurance mode: binary identity / measured identity
-> allow or deny
-> emit observability event
```

Example scenario:

```
Denied executable identity:
curl binary
```

Agent tries:

```
copy binary to /tmp/.curl2
exec that copy
```

In a high-assurance exec mode, the copied binary should still be identified as the same executable identity.

Why this is better for LLM agents

1. LLM agents are adaptive and try alternative approaches
2. LLM agents are process ecosystems rather than single binaries
3. LLM agents need dynamic policy and temporary approvals
4. agent security needs both prevention and explanation
5. deployment risk varies, so progressive assurance matters

Expanded explanation: why this architecture is better for LLM agents

1. It assumes the agent is workaround-seeking

A normal process often just fails when denied.

An LLM agent may:

- search docs
- try another path
- try another process
- try another interface
- ask for the control to be disabled

The proposed design is better because it reduces the number of weak intermediate representations the agent can exploit.

2. It models the agent as a session, not just an executable

This is a better fit for:

- shell-based workflows
- interpreter-heavy workflows
- subprocess trees
- temporary human approvals

3. It preserves product usability while strengthening enforcement

This is a critical difference from simply pushing everything into a static MAC profile.

The proposed design keeps:

- approvals
- audit
- dashboards
- event context

while strengthening the kernel trust anchor underneath them.

Key implementation improvements

Improvement 1: move file decisions into file_open

This is the highest-value change.

Why:

- the hook sees the real file object
- canonical or inode-aware reasoning becomes possible
- the pending-handoff model can be reduced or eliminated for core file policy

Implementation direction:

1. keep tracepoints for logging
2. move primary file allow/deny into LSM logic
3. add inode-aware exact-deny support for high-value resources

Improvement 2: strengthen exec identity

Recommended modes:

1. standard path mode
2. high-assurance binary-identity mode

Why:

- path-based exec policy is the wrong long-term trust anchor for adversarial agents
- binary identity directly addresses rename/copy evasions

Engineering note:

The implementation may need tiered support depending on kernel features, IMA availability, or helper support.

Improvement 3: make policy state session-scoped

Conceptual target:

AGENT_SESSIONS:

cgroup_id -> agent_id, generation_id, enforcement_flags

FILE_POLICY:

(agent_id, generation_id, resource_key) -> allow/deny

EXEC_POLICY:

(agent_id, generation_id, exec_key) -> allow/deny

TEMP_GRANTS:

(agent_id, resource_key, expiry) -> grant metadata

Why:

- clearer reasoning
- less cross-agent coupling
- cleaner reload semantics
- easier answer to: "why was this allowed?"

Improvement 4: add seccomp as standard defense-in-depth

Block or heavily review:

- ptrace
- bpf
- mount
- setns
- unshare
- io_uring_setup

Why:

These are escape-option reducers. They shrink the search space for workaround-seeking agents.

Improvement 5: mature the network layer

Target:

- cgroup-scoped egress policy
- DNS/connect correlation
- special handling for metadata services

Example scenarios:

1. block all direct metadata-service access
2. allow only corporate artifact mirrors

3. allow outbound HTTPS only for approved package hosts

Failure-model expectations

The design should explicitly define whether subsystems fail:

- open
- closed
- monitor-only

Suggested direction:

- dashboard failure should not affect enforcement
- alerting failure should not affect enforcement
- missing advanced hook support should produce explicit downgrade signals

This is important for trust. Silent security downgrades are product failures even when they are technically graceful.

Permission requests and temporary grants

Guardian includes a human-approval path because LLM agents sometimes need time-bounded access to a resource that is normally denied.

Typical flow:

1. agent tries to read or execute something not currently allowed
2. agent receives denial or knows it needs approval first
3. agent sends a permission request to the daemon over the Unix socket
4. request includes:
 - agent name
 - resource type ('file' or 'exec')
 - resource path
 - optional justification
5. daemon creates a pending request
6. dashboard shows the request to a human operator
7. operator approves or denies and chooses a duration
8. if approved, daemon creates a temporary grant with expiry time
9. agent retries within that time window
10. background cleanup removes the grant after expiry

Example:

Agent wants:
/home/dev/.aws/credentials

Human approves:
600 seconds

Result:
the path is temporarily added to the allow state

after 600 seconds the grant is removed automatically

How a timed file grant works

For file access:

1. approval creates a temporary grant record with expires_at
2. the allowed path is inserted into the BPF allow maps
3. the agent retries the access while the grant is still live
4. the cleanup task checks grant expiry every few seconds
5. once expired, the path is removed from the allow maps

This gives Guardian something that static MAC systems usually do not provide naturally: time-bounded, per-agent runtime access.

How a timed exec grant works

For exec access:

1. approval creates a temporary exec grant with expires_at
2. the requested command path is added to that agent's in-memory exec allow list
3. the grant is tracked until expiry
4. the cleanup task later removes that command from the allow list

Current implementation note:

The runtime file-grant path is directly connected to kernel allow maps, so it changes enforcement state immediately. Exec grants are currently weaker from an enforcement-model perspective because the implementation updates in-memory exec policy state, but does not yet describe the same direct dynamic kernel-map update path as file grants. This should be treated as an area to tighten in the next architecture iteration.

Engineering implication:

A useful agent-governance product must support temporary, auditable approvals without turning every one-off need into a permanent policy change, but the report should distinguish clearly between file-grant enforcement maturity and exec-grant maturity.

Testing strategy

The architecture should be tested against agent-like adversarial behavior, not only syscall correctness.

Key test categories:

1. path alias tests

2. exec identity tests
3. session identity tests
4. policy isolation tests
5. bypass-surface tests such as io_uring

Expanded examples:

1. path alias tests
 - /proc/self/root - symlink chains - hard-link aliasing - bind mounts
2. exec identity tests
 - copy binary - rename binary - wrapper-loader execution
3. identity tests
 - launcher session registration - child-process inheritance - stale cgroup cleanup
4. multi-agent tests
 - two Python agents with conflicting policy - concurrent grants and reloads
5. exfiltration tests
 - blocked local file access followed by remote fallback attempt

Example deployment scenarios

Scenario A: developer workstation coding assistant

Best fit:

- cgroup launch mode
- file policy
- exec policy
- approvals
- monitoring plus selective enforcement

Why:

High usability is required, but governance still matters.

Scenario B: CI repair agent

Best fit:

- cgroup identity
- stronger default-deny

- reduced network egress
- seccomp enabled

Why:

The environment is more controlled, so stronger defaults are acceptable.

Scenario C: infrastructure-affecting agent

Best fit:

- stronger exec identity
- strong network restrictions
- baseline AppArmor/SELinux
- optional isolated runtime tier

Why:

Impact radius is larger, so shared-host trust may be insufficient.

Final Engineering Assessment

Guardian Shell has the right product shape for LLM-agent governance, but its security core still needs to evolve toward stronger identity and stronger enforcement locality.

The most important engineering decisions are:

1. make cgroup-backed session identity primary
2. move file and exec decisions deeper into LSM-first logic
3. add seccomp as standard hardening
4. isolate policy per session
5. preserve the current rich observability and approval plane

If those changes are made, the architecture will be substantially better aligned with the actual LLM-agent threat model.

References

Internal material used:

- docs/architecture-analysis.md

- docs/guardian-shell-vs-veto-comparison.md
- docs/security-improvements-research.md
- docs/sandboxing-deep-dive.md
- docs/technical-comparison.md
- AGENT_IDENTITY.md
- implementation in guardian-ebpf/src/main.rs, guardian/src/main.rs, guardian/src/config.rs, and guardian-launch/src/main.rs

External references checked:

- Ona, "Introducing Veto: security for the next era of software"

<https://ona.com/stories/introducing-veto-security-for-the-next-era-of-software>

- Linux kernel docs, "LSM BPF Programs"

https://docs.kernel.org/bpf/prog_lsm.html

- eBPF Docs, bpf_d_path

https://docs.ebpf.io/linux/helper-function/bpf_d_path/

- Linux kernel docs, "Landlock"

<https://docs.kernel.org/userspace-api/landlock.html>

- Ubuntu documentation, "AppArmor"

<https://ubuntu.com/server/docs/how-to/security/apparmor/>

- Red Hat documentation, "Using SELinux"

https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/using_selinux/using_selinux